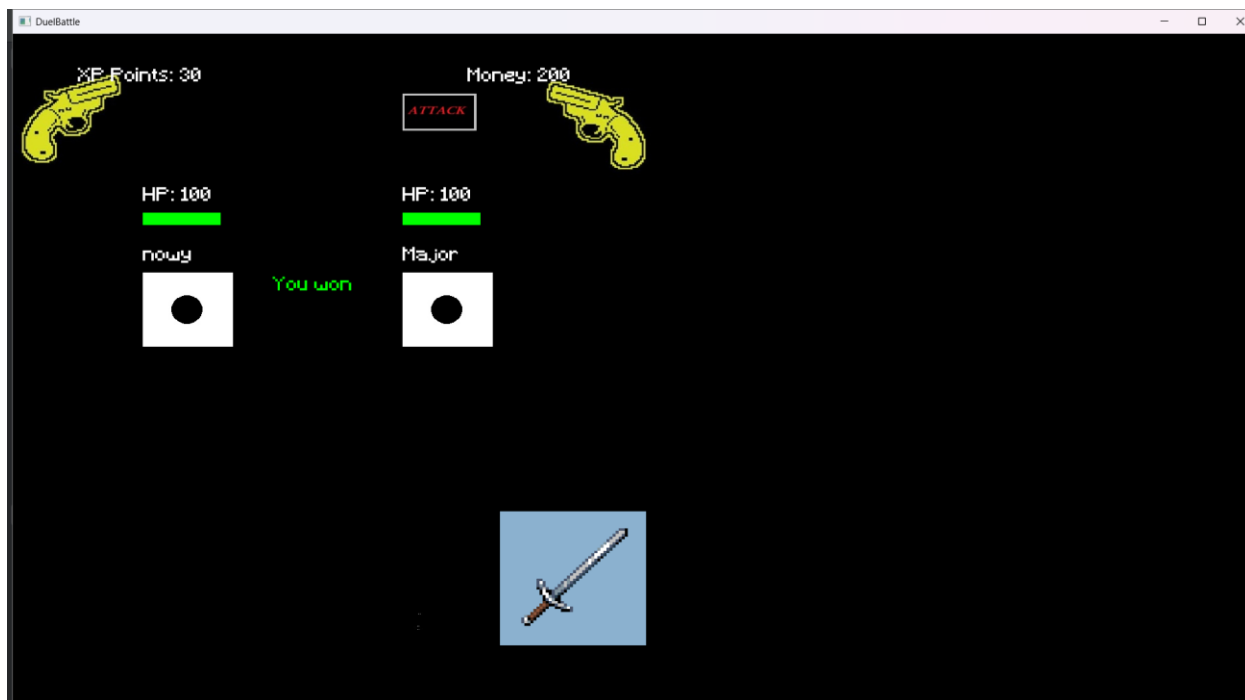
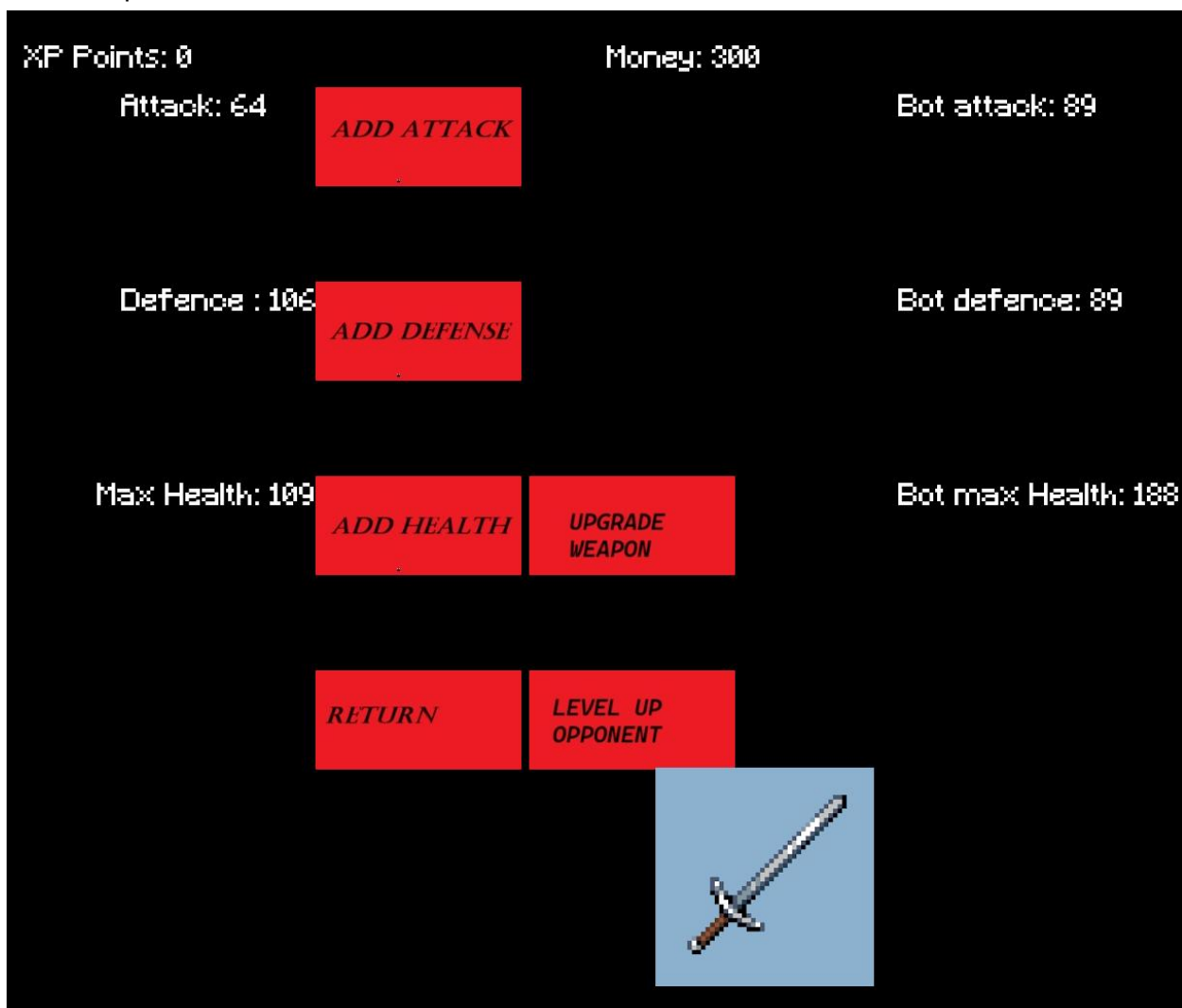


Poniżej przedstawiono wstępny wygląd interfejsu w postaci zrzutów ekranu:

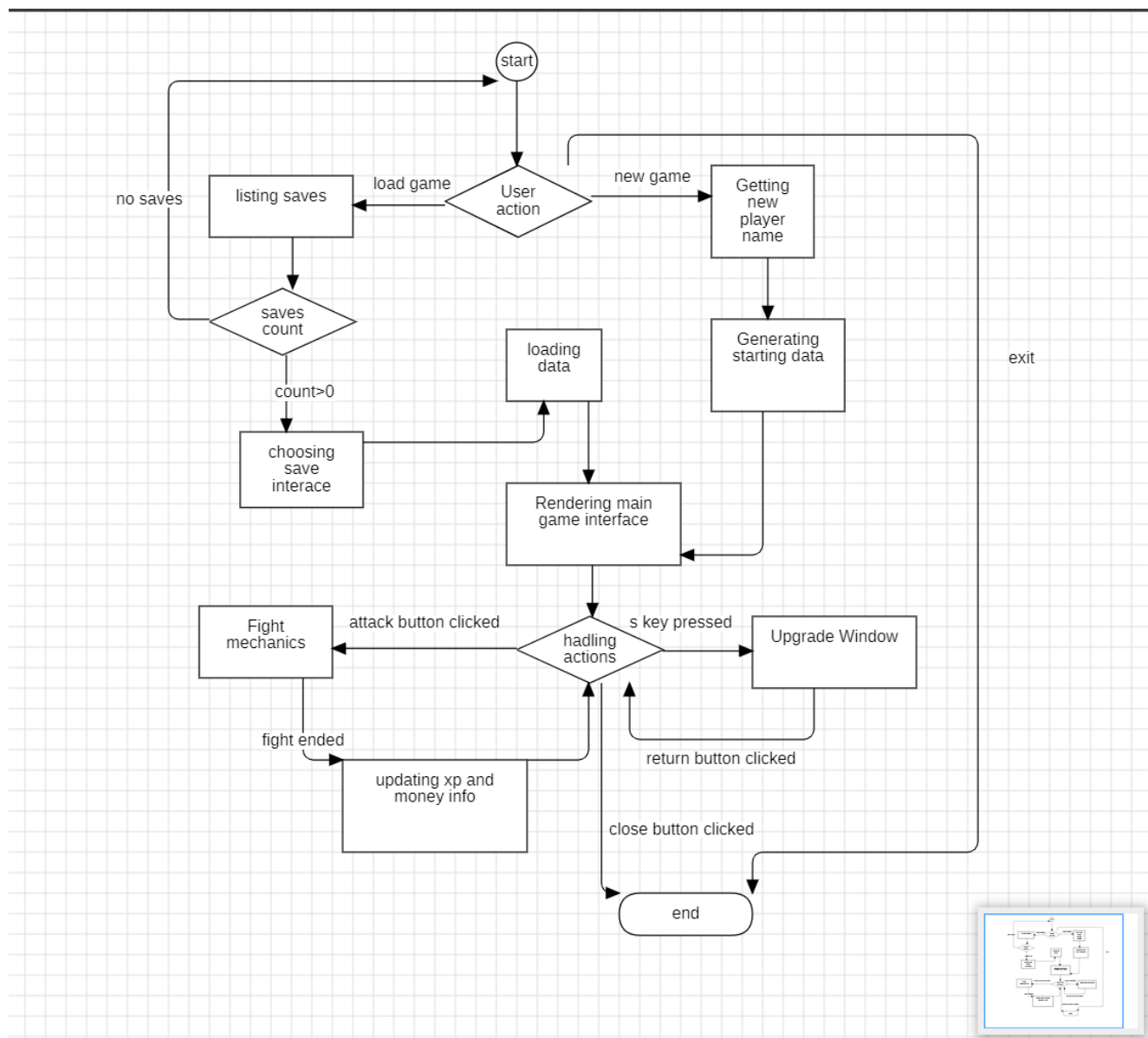
Okno walki , po prawej stronie będzie się wyświetlać animacja ataków



Okno ulepszeń:

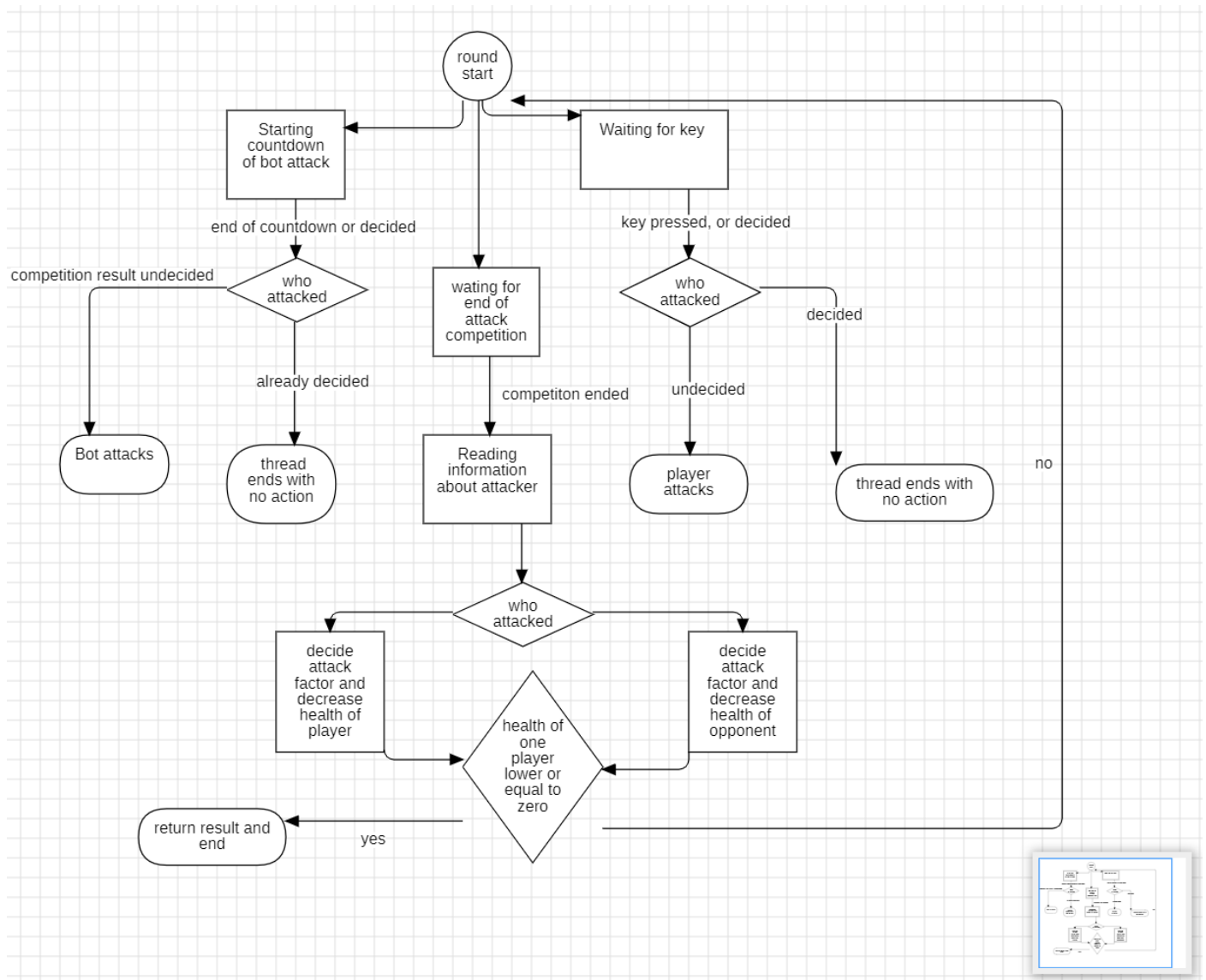


Funkcje oferowane przez grę użytkownikowi przedstawiono na schemacie blokowym



(Schemat 1)

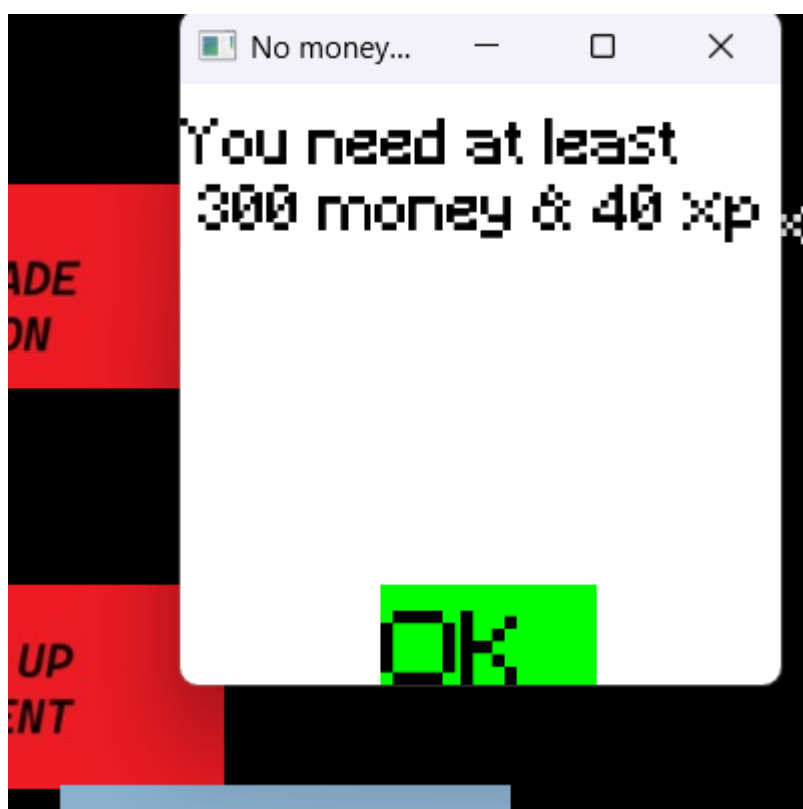
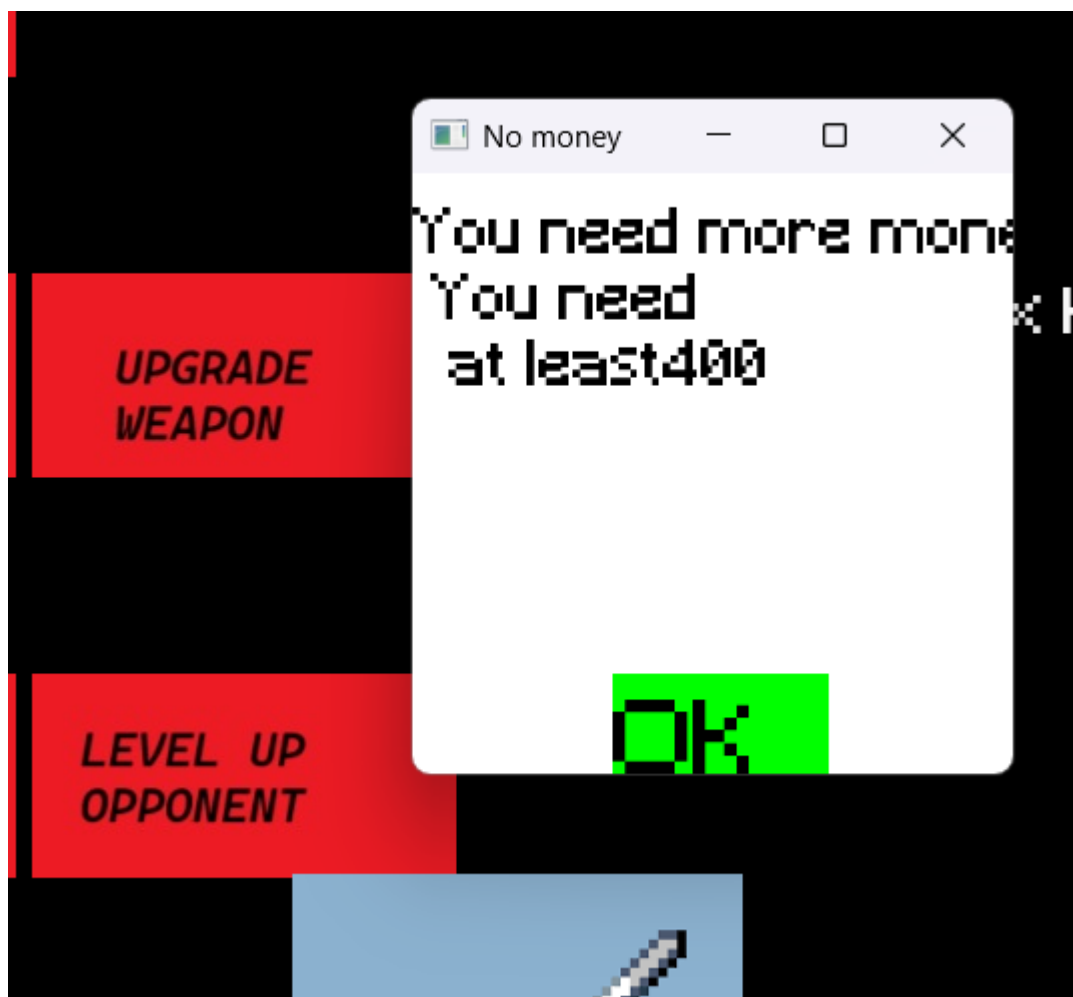
Mechanika walki została przedstawiona oddzielnie:



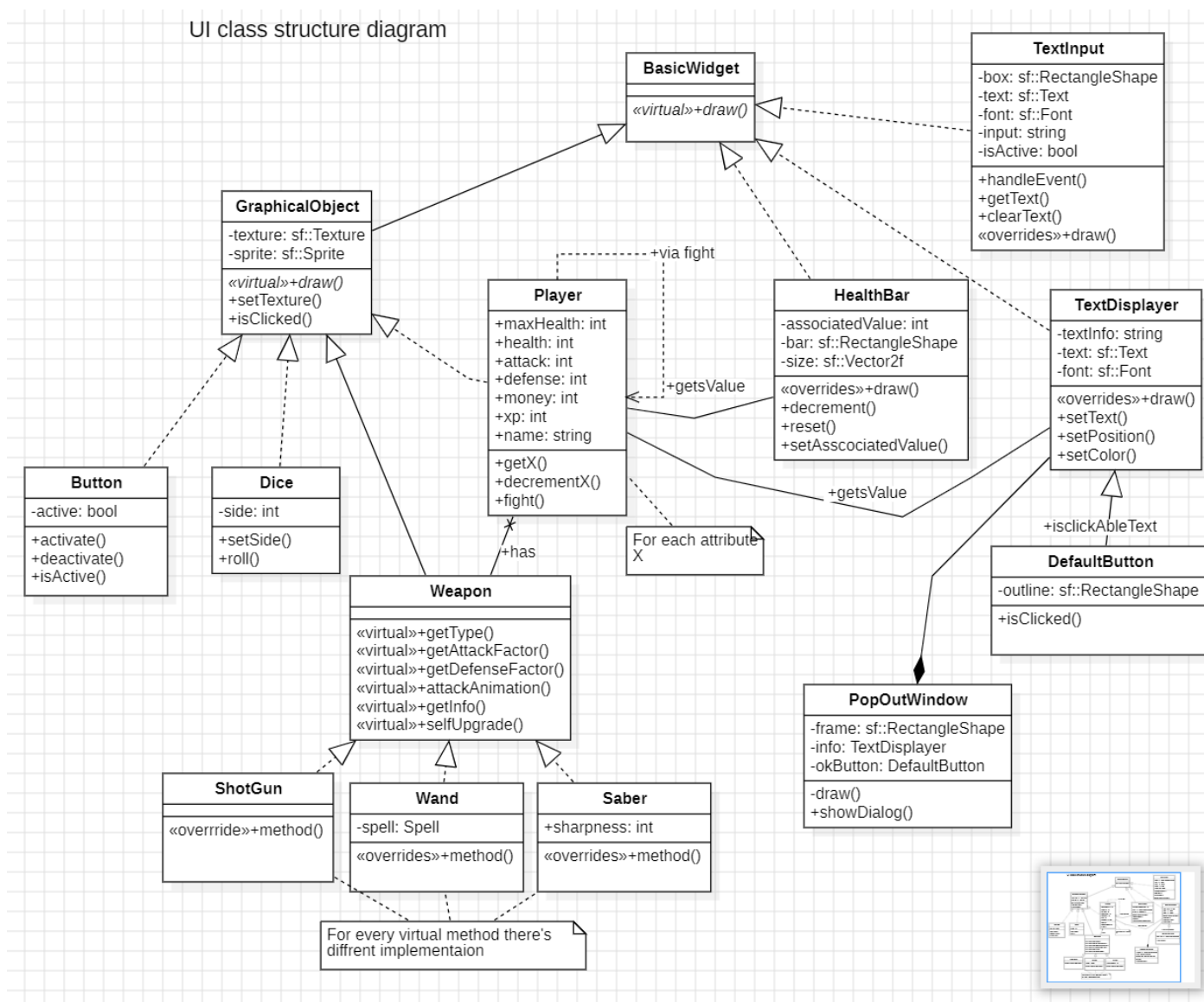
(Schemat 2)

Opis transakcji w oknie ulepszeń:

- 1.Level up przeciwnika kosztuje punkty xp
- 2.Inkrementacja pojedynczego atrybutu gracza kosztuje punkty xp
- 3.Ulepszenie broni wymaga wirtualnych pieniędzy
- 4.Transakcja może zakończyć się sukcesem lub niepowodzeniem. W przypadku niepowodzenia wyświetla się okienko z informacją :



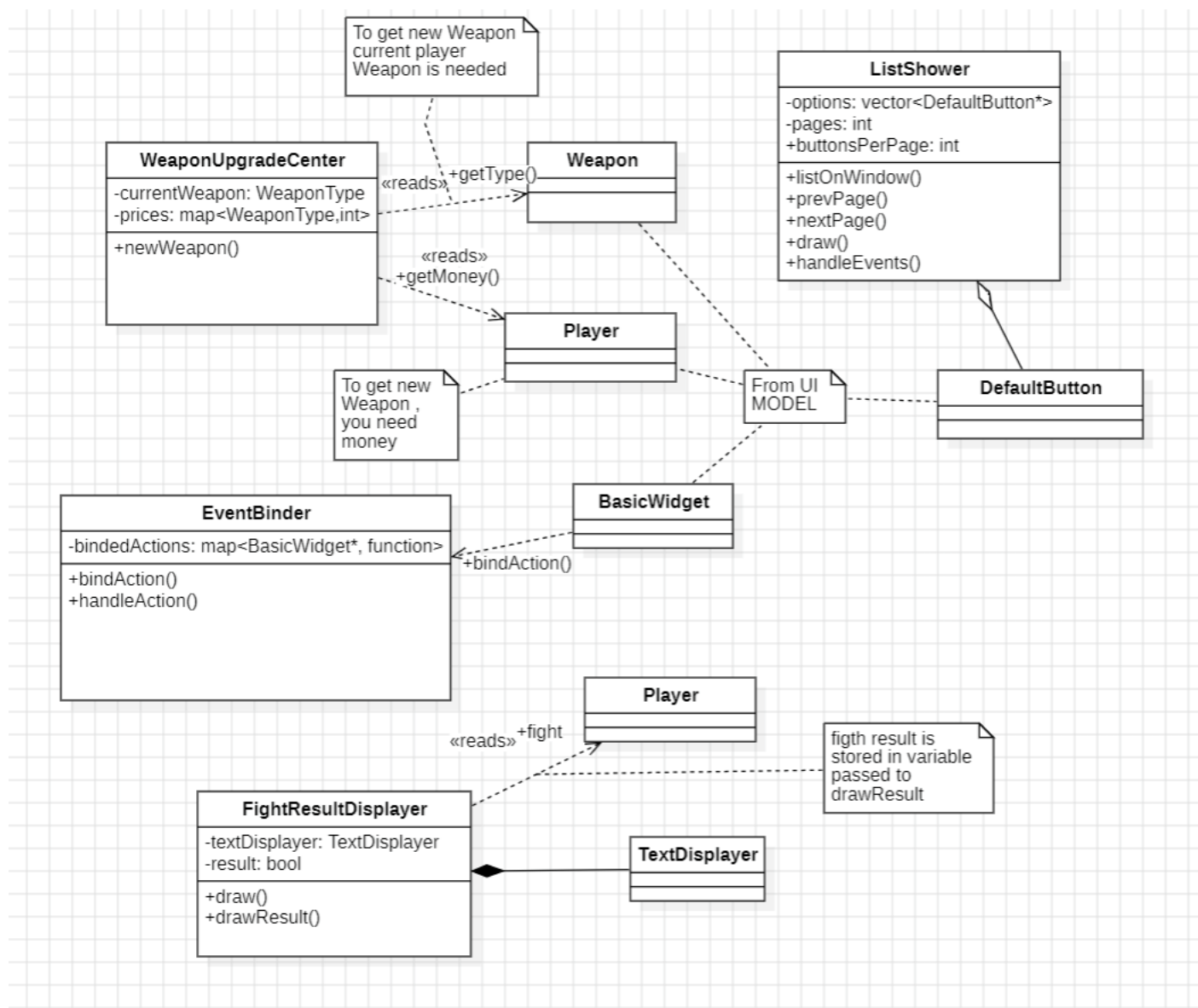
Na diagramie poniżej została przedstawiona hierarchia klas odpowiedzialnych za grafikę w projekcie:
(strzałka w złą stronę)



(Diagram 1)

Metoda selfUpgrade umożliwia zróżnicowanie mechaniki ulepszeń broni. Broń może być ulepszana pod względem własnych statystyk natomiast po osiągnięciu maksymalnego poziomu własnych statystyk funkcja selfUpgrade zwraca wartość false oznaczającą niemożność dalszego ulepszania i odblokowującą przejście na inny rodzaj broni. Dwie instancje klasy Player komunikują się między sobą poprzez metodę fight , która umożliwia zmianę punktów zdrowia oraz komunikację , że punkty zdrowia przeciwnika osiągnęły poziom 0 co jest oznaczane jako zwrócona wartość true.

W projekcie występują również klasy wspomagające obiekty graficzne , lub wykorzystujące je w bardziej złożony sposób. Przedstawia to diagram poniżej:

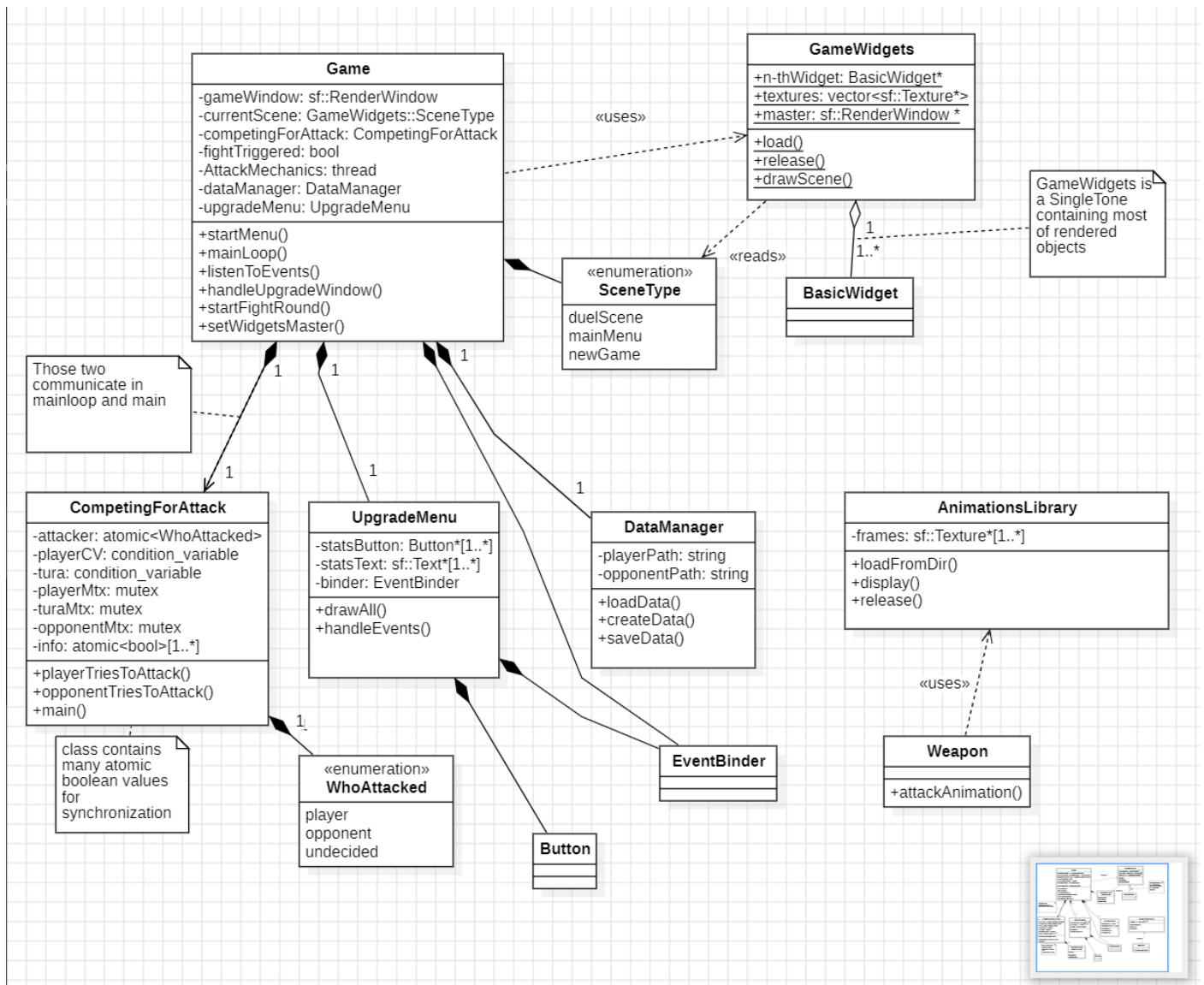


(Diagram 2)

Klasa ListShower odpowiada za pobranie i wyświetlenie savów do wybrania przez użytkownika.

WeaponUpgradeCenter wspomaga rozbudowany mechanizm ulepszania broni.

Poniżej przedstawiono diagram centralnych klas, na których oparta jest mechanika gry:



(Diagram 3)

Klasa `CompetingForAttack` opakowuje logikę współbieżnego wyścigu o pierwszeństwo ataku.

Metoda `Game::startFightRound` uruchamia wewnętrzną pętlę walki i komunikuje się z z klasą `CompetingForAttack` poprzez pola tura, `fightTriggered` wraz z innymi zmiennymi boolowskimi.

Instancja klasy DataManager jest wykorzystywana do pobierania, zapisywania i tworzenia danych związanych z graczem. Klasa AnimationsLibrary tworzy interfejs ładowania (loadFromDir) i wyświetlania animacji (display). Jest ona używana przez klasy dziedziczące po Weapon do wyświetlania animacji walki w metodzie attackAnimation.

Klasa `GameWigets` przechowuje poprzez dynamiczną alokację obiekty i dane, do których odwołują się metody innych obiektów w celu rozdzielenia logiki. Na przykład metoda `Game::startFightRound` odwołuje się do instancji klasy `Player`, klasy `Dice`, oraz kilku instancji klasy `TextDisplayler` i klasy `HealthBar` celem odświeżania informacji o graczach oraz wizualizacji przebiegu walki.

Metody `Dice::roll` wykorzystują statyczną metodę `GameWidgets::drawScene` w celu wyświetlenia pseudo animacji rzutu kością. Również `AnimationsLibrary::display` potrzebuje wyświetlić wszystkie obiekty graficzne związane z oknem walki i wykorzystuje metodę `GameWidgets::drawScene`.